

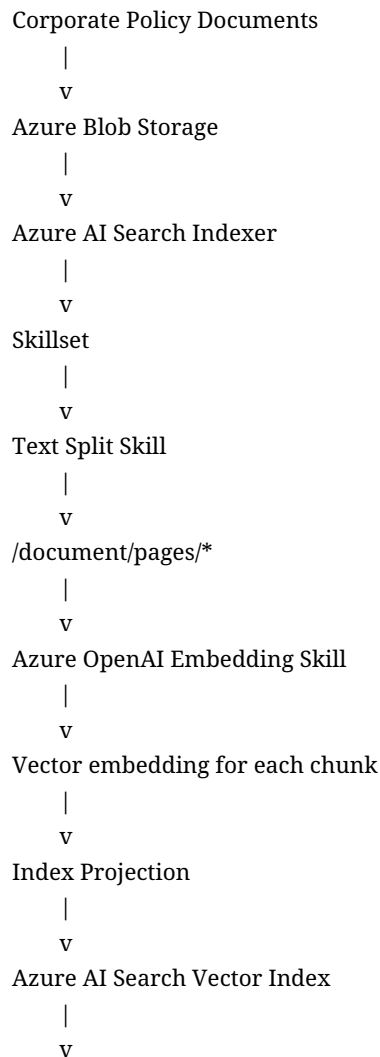
Azure AI Search - Embeddings Skill Implementation Strategy

Corporate Policy Assistant RAG Architecture

1. Purpose

This strategy defines how to configure Azure AI Search so that text chunks produced by the existing Text Split skill are converted into vector embeddings and stored in the Corporate Policy Assistant search index.

The target implementation uses Azure AI Search integrated vectorization: the indexer invokes the skillset, the Text Split skill creates chunks, an Azure OpenAI Embedding skill vectorizes each chunk, and index projections write chunk-level records into the vector-enabled search index.



Hybrid / Vector Retrieval
|
v
Policy Assistant LLM

2. Current State

Component	Current State
Azure AI Search service	Implemented
Blob data source	Implemented
Search index	Implemented
Indexer	Implemented
Custom skillset	Implemented
Text Split skill	Implemented
Text Split output	/document/pages/*
Chunking mode	pages
Chunk unit	azureOpenAITokens
Chunk size	1000 tokens
Chunk overlap	125 tokens
Embedding model	text-embedding-3-small deployed
Azure OpenAI Embedding skill	To be implemented
Index projection	To be verified/configured
Query-time vectorizer	To be verified/configured

3. Implementation Objective

Implement an Azure OpenAI Embedding Skill inside:

as-corp-policy-ai-rag-skillset-dev

The embedding skill should:

1. Receive each text chunk from /document/pages/*.
2. Send each chunk to the deployed text-embedding-3-small model.
3. Generate a vector for every chunk.
4. Store the resulting embedding in the Azure AI Search enrichment tree.
5. Project chunk text, embedding, document identifier, title, and useful metadata into the search index.
6. Use the same embedding model at query time so indexed policy chunks and user questions are represented in the same vector space.

4. Recommended Naming Convention

Artifact	Recommended Name
Skillset	as-corp-policy-ai-rag-skillset-dev
Text Split Skill	as-corp-policy-ai-rag-skillset-text-split-dev

Embedding Skill	as-corp-policy-ai-rag-skillset-embedding-dev
-----------------	--

5. Phase 1 - Verify the Embedding Deployment

Before modifying the skillset, verify the deployed embedding model and record:

- Resource URI
- Deployment ID
- Model name
- Embedding dimensions
- Authentication method

Recommended baseline:

Model Name: text-embedding-3-small
Dimensions: 1536

Resource URI:
`https://<your-foundry-or-azure-openai-resource-endpoint>/`

Deployment ID:
`<your-actual-embedding-deployment-name>`

Important: Do not assume that the deployment ID is the same as the model name. Use the deployment name actually configured in Azure.

6. Phase 2 - Choose Embedding Dimensions

For the initial Corporate Policy Assistant implementation, use 1536 dimensions for text-embedding-3-small. This keeps the first implementation simple and avoids introducing dimension reduction before retrieval quality has been validated.

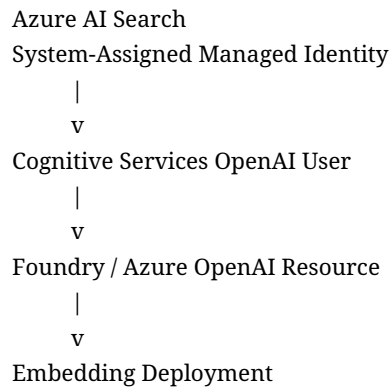
Embedding Skill dimensions
=
Search Index vector field dimensions
=
Query-time vectorizer/model configuration

Recommended baseline: 1536

7. Phase 3 - Configure Authentication

Option A - Managed Identity

Recommended for the target architecture. Use the Azure AI Search service managed identity and grant it the least-privileged role required to invoke the embedding deployment. For Azure OpenAI embeddings, this is typically the Cognitive Services OpenAI User role.



Option B - API Key

An API key can be used for initial development, but managed identity is preferred for a production-oriented implementation because it avoids storing credentials in the skillset configuration.

8. Phase 4 - Add the Azure OpenAI Embedding Skill

In the Azure AI Search skillset editor:

7. Open as-corp-policy-ai-rag-skillset-dev.
8. Select Add new skill.
9. Choose Azure OpenAI Embedding Skill.
10. Set the skill context to /document/pages/* so it runs once per chunk.
11. Set the text input source to /document/pages/*.
12. Set the output target name to vector.
13. Configure the deployment resource URI, deployment ID, model name, dimensions, and authentication.

Recommended logical mapping:

```
Context:    /document/pages/*
Input name: text
Input source: /document/pages/*
Output name: embedding
Target name: vector
Model:      text-embedding-3-small
Dimensions: 1536
```

9. Recommended Embedding Skill JSON

```
{
  "@odata.type": "#Microsoft.Skills.Text.AzureOpenAIEmbeddingSkill",
  "name": "as-corp-policy-ai-rag-skillset-embedding-dev",
  "description": "Generates vector embeddings for corporate policy text chunks.",
  "context": "/document/pages/*",
  "resourceUri": "https://<YOUR-RESOURCE-ENDPOINT>/",
  "deploymentId": "<YOUR-EMBEDDING-DEPLOYMENT-NAME>",
```

```

"modelName": "text-embedding-3-small",
"dimensions": 1536,
"inputs": [
  {
    "name": "text",
    "source": "/document/pages/*"
  }
],
"outputs": [
  {
    "name": "embedding",
    "targetName": "vector"
  }
]
}

```

10. Phase 5 - Combine Text Split and Embedding Skills

The skillset should execute in this logical sequence:

```

/document/content
|
v
Text Split Skill
|
v
/document/pages/*
|
v
Embedding Skill
|
v
/document/pages/*/vector

```

11. Phase 6 - Verify the Search Index Vector Field

Before running the indexer, inspect the target index and verify that it contains a vector field capable of storing the generated embedding.

```

{
  "name": "contentVector",
  "type": "Collection(Edm.Single)",
  "searchable": true,
  "dimensions": 1536,
  "vectorSearchProfile": "<vector-profile-name>"
}

```

The vector field dimension must match the Embedding skill dimension.

12. Phase 7 - Configure Index Projections

For a chunk-based RAG design, index projections should create separate searchable records for each chunk while preserving metadata from the source policy document.

Chunk ID	Parent ID	Title	Chunk	Vector
001	policy-01	Remote Work Policy	Employees may...	[...]
002	policy-01	Remote Work Policy	Managers must...	[...]
003	policy-01	Remote Work Policy	Exceptions...	[...]

13. Recommended Fields for the Policy RAG Index

- chunk_id
- parent_id
- title
- content
- content_vector
- source_file
- source_path
- policy_category
- policy_name
- effective_date
- last_modified

Minimum RAG fields: chunk_id, parent_id, title, content, and content_vector.

14. Phase 8 - Add the Query-Time Vectorizer

The Embedding Skill operates at indexing time. The vectorizer operates at query time. The same embedding model should be used for both so that policy chunks and user questions are represented in the same vector space.

```
INDEXING
Policy chunk
|
v
text-embedding-3-small
|
v
1536-dimensional vector

QUERY
User question
|
```

v
text-embedding-3-small
|
v
1536-dimensional query vector

15. Phase 9 - Verify the Indexer

The indexer should reference:

```
"skillsetName": "as-corp-policy-ai-rag-skillset-dev"
```

Also verify that `targetIndexName` points to the chunk/vector-enabled target index.

16. Phase 10 - Test Before Full Reindexing

- [] Embedding skill saves successfully.
- [] `resourceUri` is correct.
- [] `deploymentId` exactly matches the Azure deployment name.
- [] `modelName` is `text-embedding-3-small`.
- [] `dimensions` is 1536.
- [] Embedding input source is `/document/pages/*`.
- [] Vector output target is defined.
- [] The index contains a compatible vector field.
- [] Vector field dimensions equal 1536.
- [] Index projection maps individual chunks into index records.
- [] Indexer references `as-corp-policy-ai-rag-skillset-dev`.
- [] Azure AI Search identity can call the embedding deployment.

17. Phase 11 - Reset and Run the Indexer

After the complete enrichment pipeline has been validated, reprocess existing policy documents in the following order:

Indexer
|
v
Reset
|
v
Run
|
v
Refresh

Reset marks the existing corpus for reprocessing; Run executes the indexer; Refresh updates the portal view/status.

18. Phase 12 - Validate Indexed Results

After the indexer succeeds, use Search Explorer to verify:

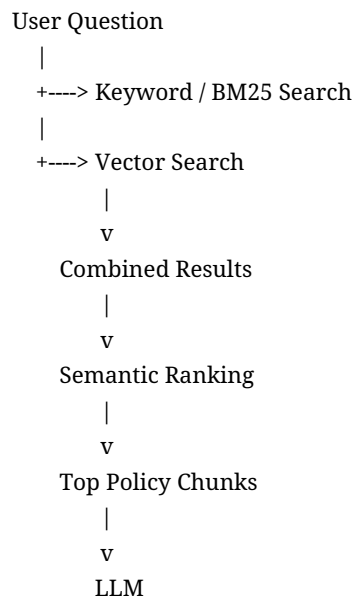
- Each source policy produces multiple chunk-level records when appropriate.
- Each chunk contains the expected chunk text.
- Each chunk includes a populated vector field.
- Parent/source metadata is retained.
- No dimension, authorization, or enrichment errors are reported.

19. Phase 13 - Test Vector and Hybrid Retrieval

Test representative policy questions, for example:

- What is the corporate remote work policy?
- How frequently do employees need to change passwords?
- What is the incident escalation procedure?
- What policies apply to cloud costs?

For the target RAG architecture, prefer hybrid retrieval so keyword/BM25 and vector similarity can complement one another.



20. Security Strategy

Recommended target-state security:

- Use Azure AI Search managed identity.
- Grant only the role required to call the embedding model.
- Avoid broad Contributor/Owner permissions for runtime embedding calls.
- Prefer RBAC over embedded API keys for production.

- Keep Azure Blob Storage private and use identity-based access where practical.

21. Cost and Performance Considerations

The main variable cost associated with this stage comes from embedding-model usage. Cost is influenced by the number of source documents, the number of chunks generated per document, chunk size, overlap, and how often the corpus is reindexed.

Approximate cost drivers:

Number of source documents

x

Chunks per document

x

Tokens per chunk

x

Embedding operations

Keep the current 1000-token / 125-token overlap configuration for the first end-to-end baseline. After the pipeline works, compare retrieval quality and cost against a smaller configuration such as 512 tokens / 64 overlap.

22. Error Conditions to Watch For

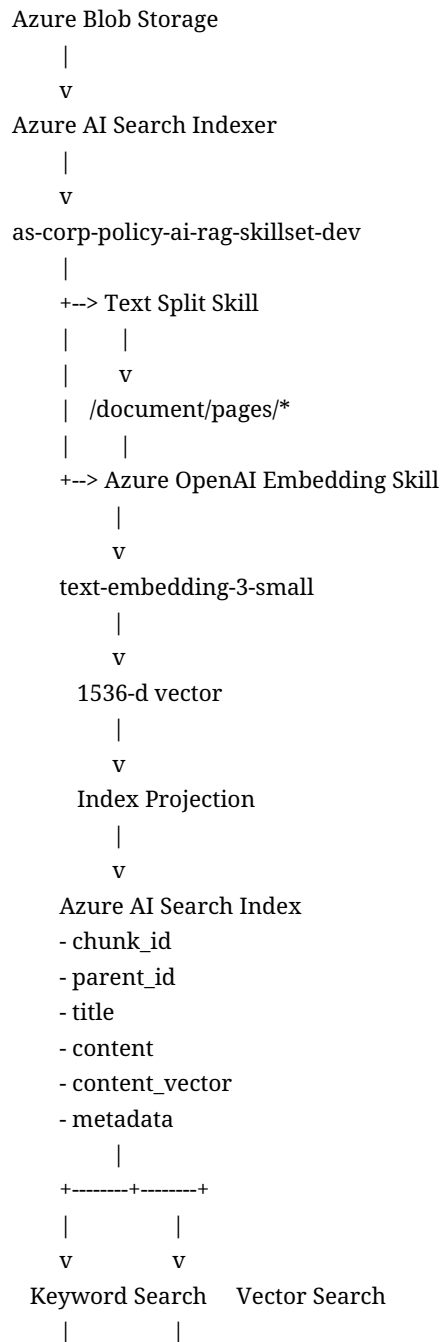
Error / Symptom	Primary Checks
400 - Invalid skill configuration	@odata.type, deploymentId, modelName, resourceUri, inputs, outputs
401 / 403 - Authorization failure	Search managed identity, RBAC role, API key configuration
429 - Rate limit	Embedding deployment TPM/RPM limits and indexer throughput
Vector dimension error	Embedding dimensions must equal index vector field dimensions
Empty embeddings	Verify /document/pages/* is populated and the Text Split skill succeeds

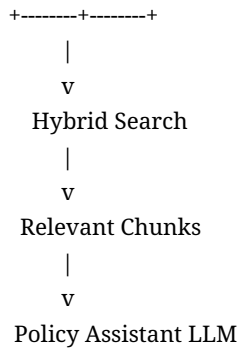
23. Recommended Implementation Sequence

14. Verify text-embedding-3-small deployment.
15. Record resource URI and deployment ID.
16. Verify authentication / managed identity.
17. Add Azure OpenAI Embedding Skill.
18. Set input to /document/pages/*.
19. Set output target to vector.
20. Set dimensions to 1536.
21. Verify index vector field dimensions are 1536.

22. Configure index projection.
23. Configure or verify query-time vectorizer.
24. Verify indexer skillsetName.
25. Reset indexer.
26. Run indexer.
27. Check indexer results and errors.
28. Test vectors in Search Explorer.
29. Test hybrid policy retrieval.

24. Recommended Target Architecture





25. Immediate Next Action

The next action should be to configure the Azure OpenAI Embedding Skill, not to reset or run the indexer yet.

30. Open the deployed text-embedding-3-small model.
31. Record the deployment name, resource URI/endpoint, dimensions, and authentication method.
32. Return to as-corp-policy-ai-rag-skillset-dev.
33. Select Add new skill.
34. Add Azure OpenAI Embedding Skill.
35. Configure it to consume /document/pages/* and output vector.
36. Save and validate the skill.
37. Then validate the index projection and vector field before reindexing.

Implementation sequence: Text Split -> Embedding Skill -> Index Projection -> Vector Index Validation -> Query Vectorizer -> Reset/Run -> Retrieval Testing